

Approach

Understanding the Task

- The task is to fetch data from public APIs.
- We need to get: Blog posts from /posts, Todo items from /todos
- Then display them on the webpage.
- We also need to handle errors properly.
- Finally, we must use a JavaScript design pattern to organize the code.

Using JSONPlaceholder API

- I used the public API: <https://jsonplaceholder.typicode.com/>
- It provides fake online data for testing.
- I used two endpoints:
 - /posts → to get blog posts
 - /todos → to get todo list items

Creating HTML Structure

- Created two sections: One for Blog Posts, One for Todos
- Added two buttons: "Load Posts", "Load Todos"
- Created two div containers: blogContainer, todoContainer
- These containers are used to display the fetched data.

Using Fetch API

- Used fetch() to request data from the server.
- Used async and await for asynchronous programming.
- Converted response into JSON using:
- Created a reusable function getServerData() to fetch data from any URL.

Displaying Data on Webpage

- Used DOM methods like: document.createElement(), appendChild()
- For posts: Displayed title and body.
- For todos: Displayed title, Displayed status (Completed or Pending)
- Used .slice(0,5) to show only first 5 items for better display.

Error Handling

- Checked if data is an array.
- Used try...catch block to catch errors.
- Displayed error messages on the webpage.
- Showed user-friendly message if server fails or internet is not working.

Using JavaScript Design Pattern

- Used Revealing Module Pattern.
- Wrapped the entire code inside an IIFE (Immediately Invoked Function Expression).
- Kept variables private inside the module.
- Exposed only required functions: fetchBlogPosts, fetchTodoTasks

Integration

- Connected buttons with module functions.
- When user clicks button: Data is fetched, Loading message is shown, Data is displayed, If error occurs, error message is shown

Learning experience and any insights

This assignment helped me understand how to use the Fetch API to get data from a server and display it on a webpage. I learned how to handle errors properly using try-catch and check server responses. I also understood how using a design pattern like the Module Pattern helps organize code and make it cleaner and more maintainable.